



ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

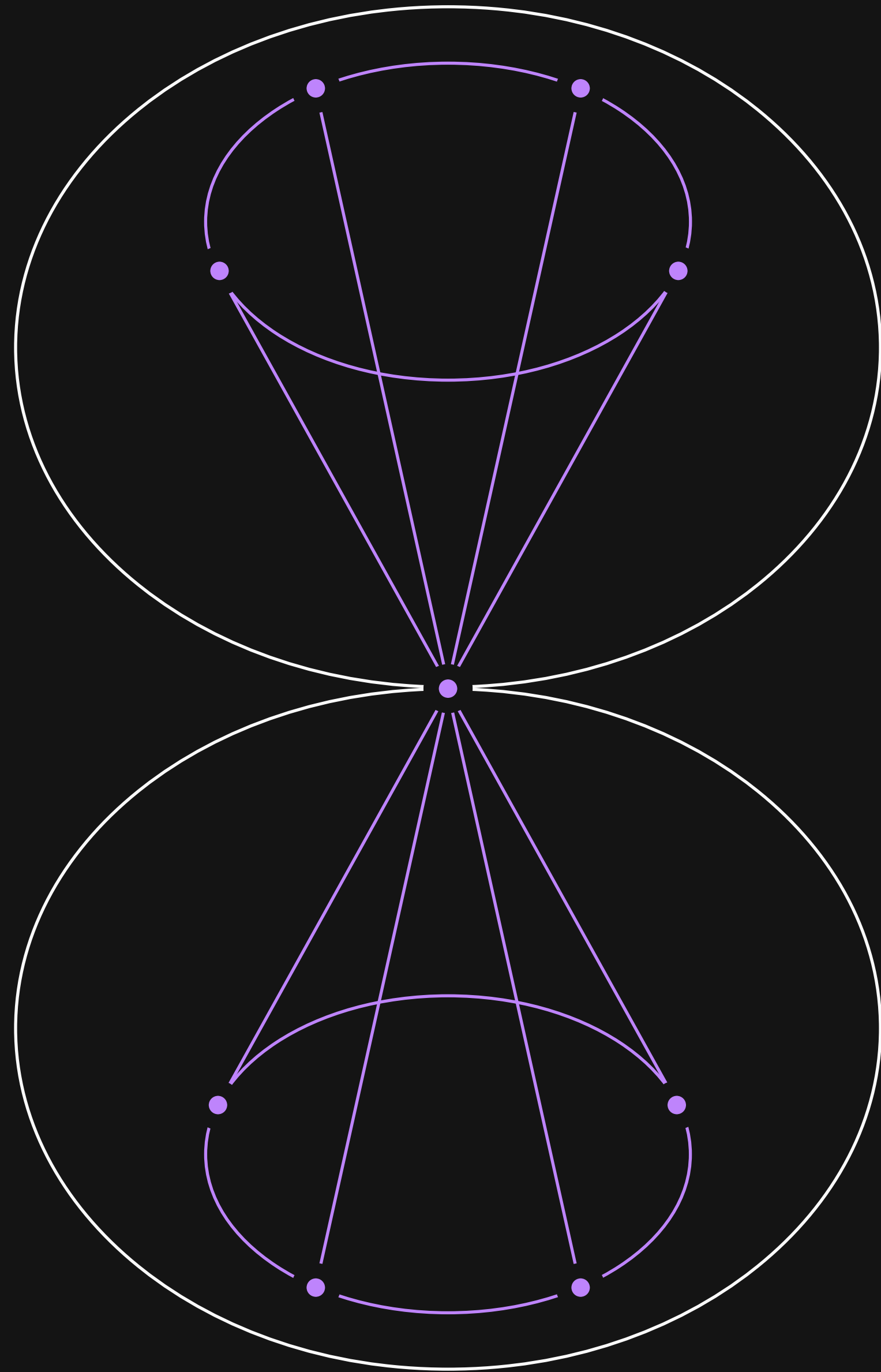
Агрегация данных

SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛЕКЦИЯ

НЕДЕЛЯ 3

План занятия



- 01 Функции агрегации
- 02 Группировка данных
- 03 Предложение HAVING
- 04 Агрегация с условиями
- 05 Скалярные подзапросы

Агрегация данных

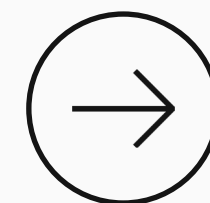
Агрегация данных



Агрегация данных — это объединение данных из нескольких строк в результирующем наборе на основе группировки.

Применяется при расчёте метрик, для анализа и создания сводных таблиц.

customer_id	order_sum
101	2000
101	5000
102	1500
103	3500
103	4000

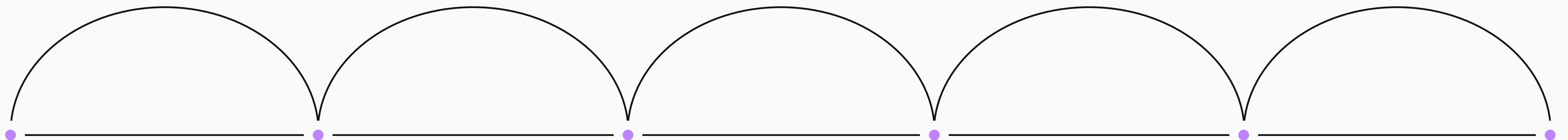


customer_id	max_order_sum
101	5000
102	1500
103	4500

Функции агрегации



Функции агрегации позволяют выполнять **сводные расчёты** по группам строк и возвращают один результат для каждой группы, например сумму, среднее или максимум.



COUNT

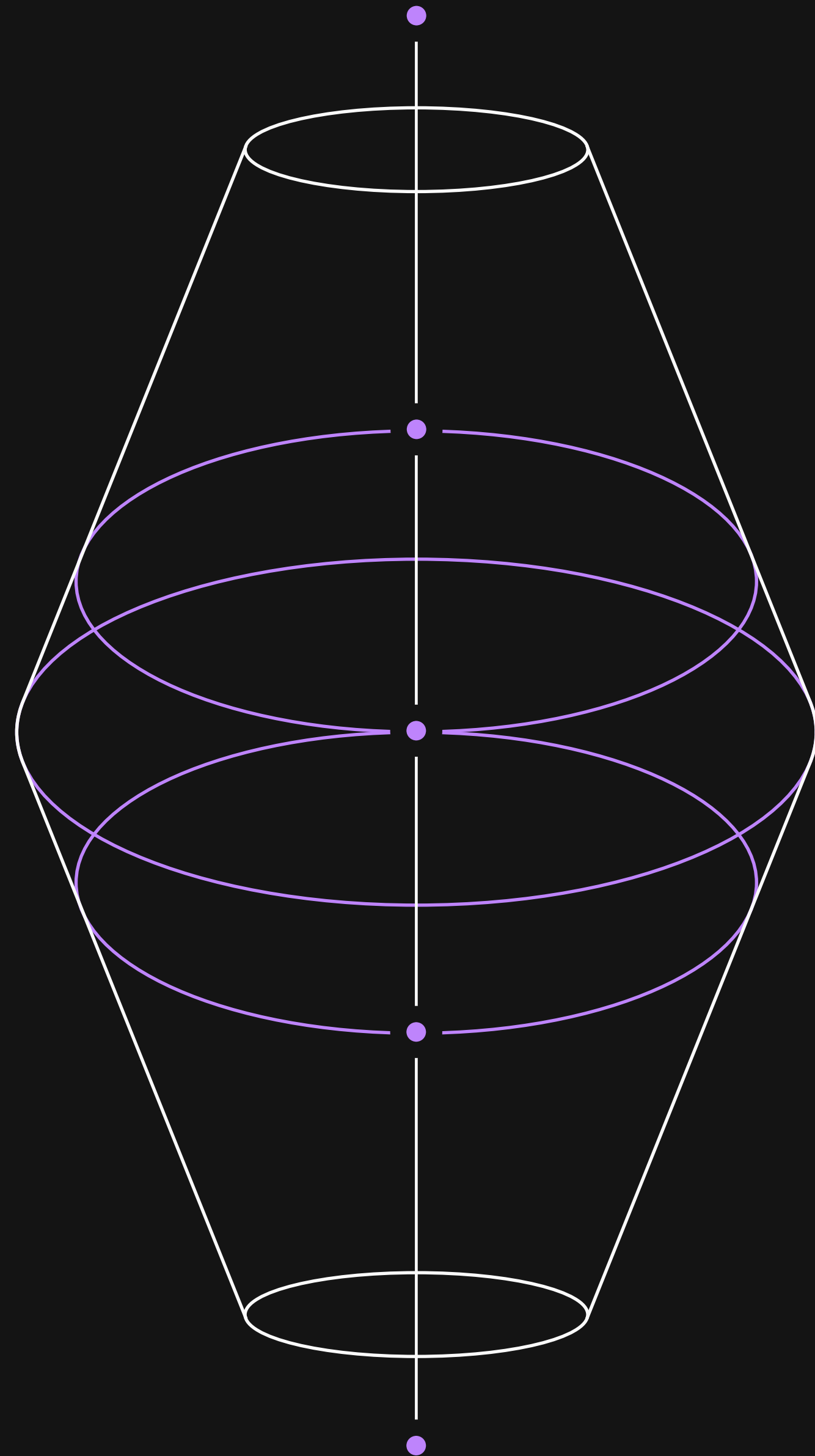
Функция подсчёта количества строк

→ Синтаксис:

- **COUNT (*)** считает все строки, включая **NULL**;
- **COUNT (column_name)** пропускает **NULL**-значения.

→ **COUNT (DISTINCT column_name)** считает все уникальные значения поля.

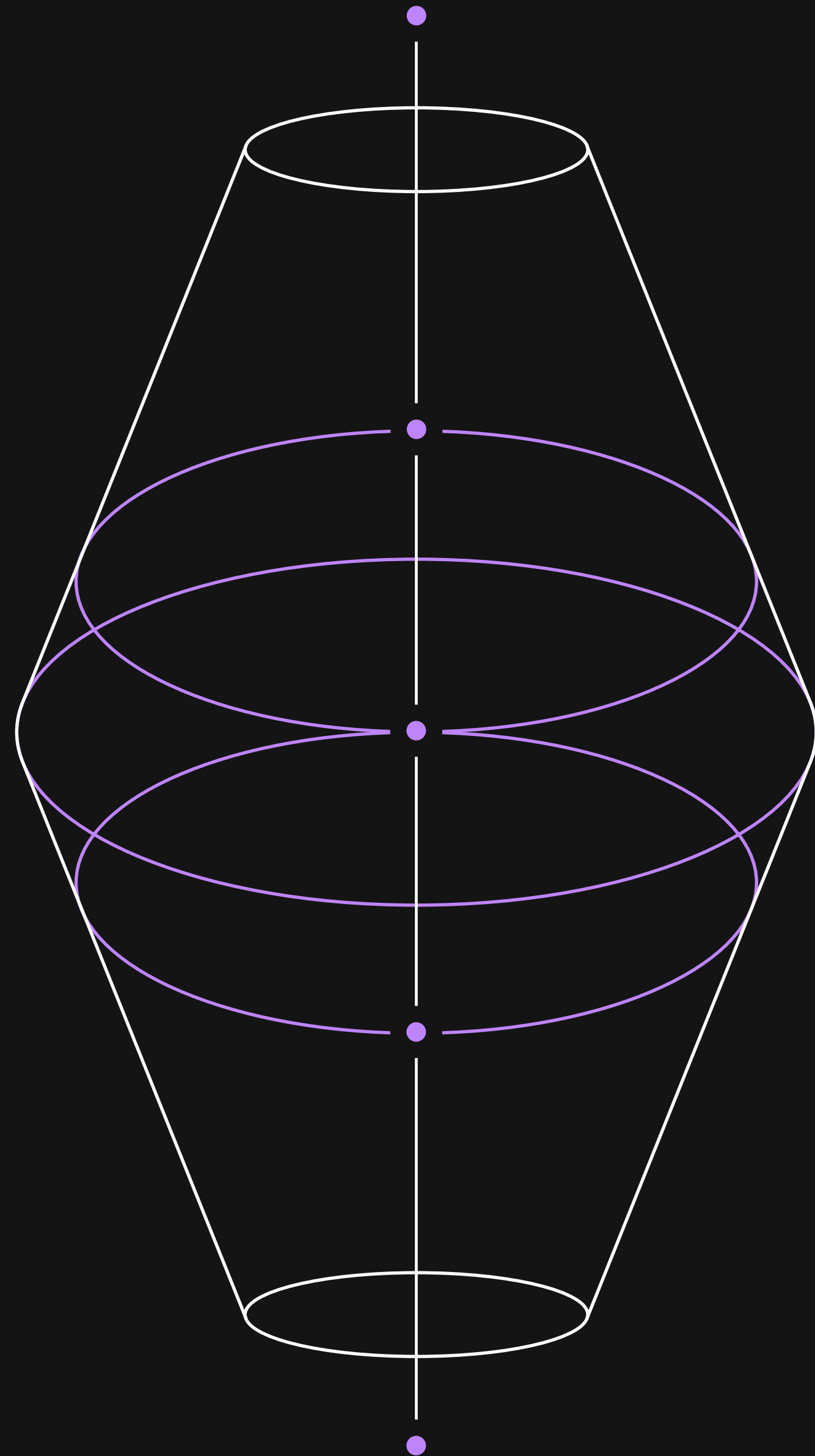
→ Если все значения поля содержат **NULL**, то функция **COUNT (column_name)** вернёт 0.



SUM

Функция подсчёта количества строк

- Синтаксис:
`SUM(column_name) as sum_column.`
- Если поле содержит **NULL**-значения, при суммировании они будут проигнорированы.
- Если все значения поля содержат **NULL**, то функция вернёт **NULL**.
- Применима только к числовым типам данных.



Количество строк и сумма



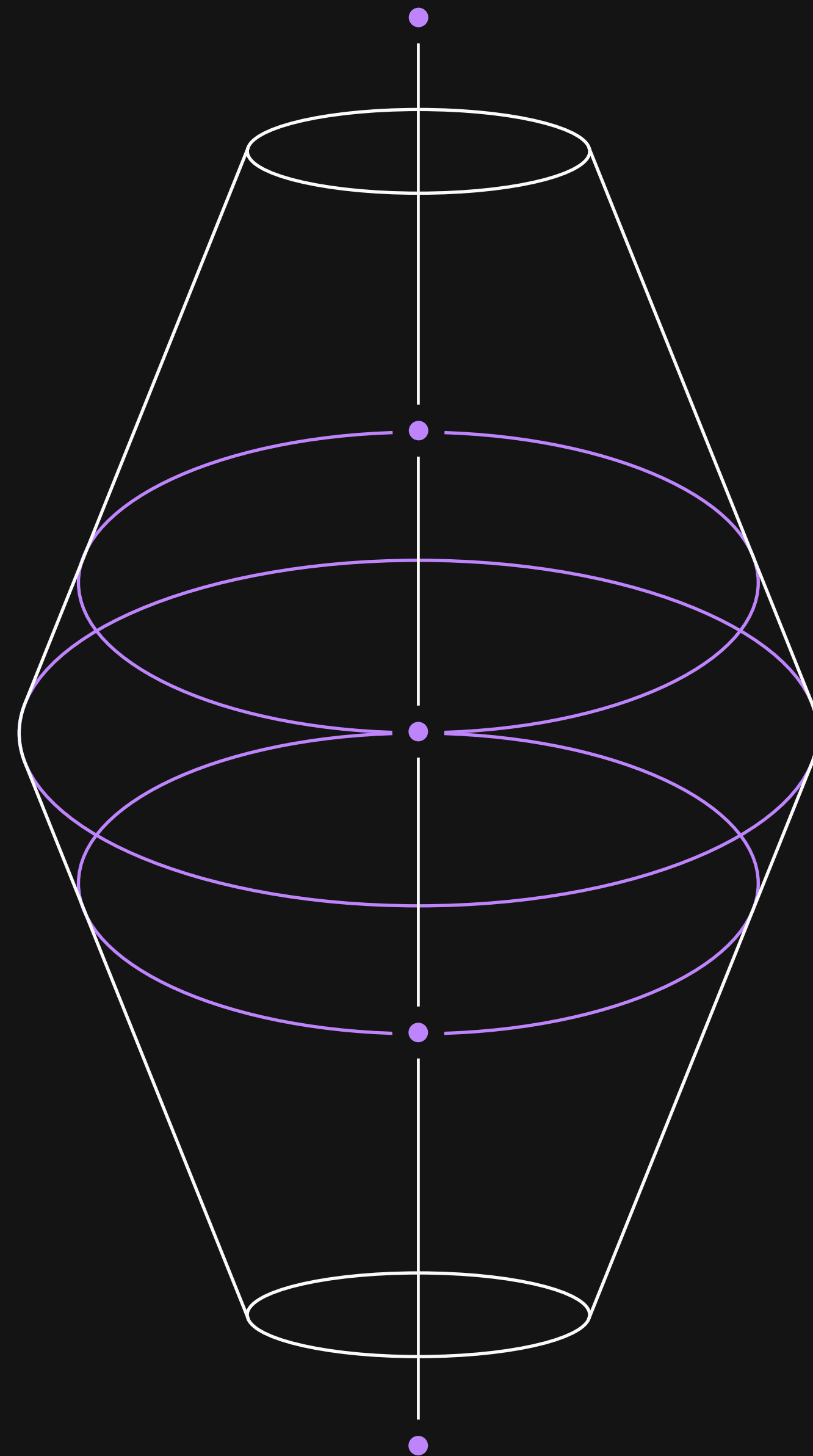
ПРИМЕР 1

По таблице с заказами посчитать сумму заказов, их количество, а также количество уникальных клиентов.

```
SELECT
    SUM(total_amount) AS total_sum,
    COUNT(*) AS orders_count,
    COUNT(DISTINCT customer_id) AS cntd_customers
FROM shop_orders
```


AVG, MIN, MAX

Функция подсчёта среднего,
минимального и максимального значений



- Синтаксис:
 - `AVG(column_name) as avg_column,`
 - `MIN(column_name) as min_column,`
 - `MAX(column_name) as max_column.`
- Если поле содержит **NULL**-значения, при подсчёте они будут проигнорированы.
- Если все значения поля содержат **NULL**, то функция вернёт **NULL**.
- **AVG** применима только к числовым типам данных, **MIN** и **MAX** — также к датам и текстовым данным.

AVG, MIN, MAX



ПРИМЕР 2

Найти среднюю, минимальную и максимальную суммы выполненного заказа.

```
SELECT
    MIN(total_amount) AS min_sum,
    MAX(total_amount) AS max_sum,
    AVG(total_amount) AS avg_sum
FROM shop_orders
WHERE order_status = 'Completed'
```

Вопрос на подумать



ПРИМЕР 3



Как найти минимальное или максимальное значение в колонке без использования агрегирующих функций?

Вопрос на подумать



ПРИМЕР 3



Как найти минимальное или максимальное значение в колонке без использования агрегирующих функций?



```
SELECT
    total_amount AS max_sum
FROM shop_orders
WHERE order_status = 'Completed'
ORDER BY total_amount DESC
LIMIT 1
```

Обработка NULL-значений

Обработка NULL-значений



ПРИМЕР 4

Найти количество доставленных заказов.

⓪ Какими способами можно найти это число?

Обработка NULL-значений



ПРИМЕР 4

Найти количество доставленных заказов.

• • •

```
SELECT
    COUNT(*) AS cnt_delivered
FROM shop_orders
WHERE delivery_date IS NOT NULL
```

Вариант решения 1

• • •

```
SELECT
    COUNT(delivery_date) AS
cnt_delivered
FROM shop_orders
```

Вариант решения 2

Обработка NULL-значений



ПРИМЕР 5

Выведи количество доставленных заказов (у которых заполнена дата доставки), общее количество заказов интернет-магазина и процент доставленных заказов.

Обработка NULL-значений



ПРИМЕР 5

Выведи количество доставленных заказов (у которых заполнена дата доставки), общее количество заказов интернет-магазина и процент доставленных заказов.

• • •

```
SELECT
    COUNT(*) AS cnt_all,
    COUNT(delivery_date) AS cnt_delivered,
    ROUND(COUNT(delivery_date) * 100.0 / COUNT(*))::int AS delivered_prc
FROM shop_orders;
```

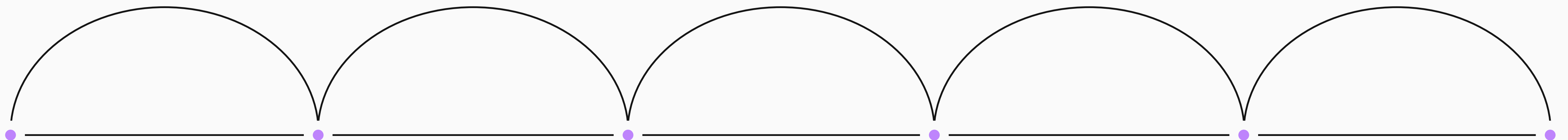
Группировка данных

Группировка данных



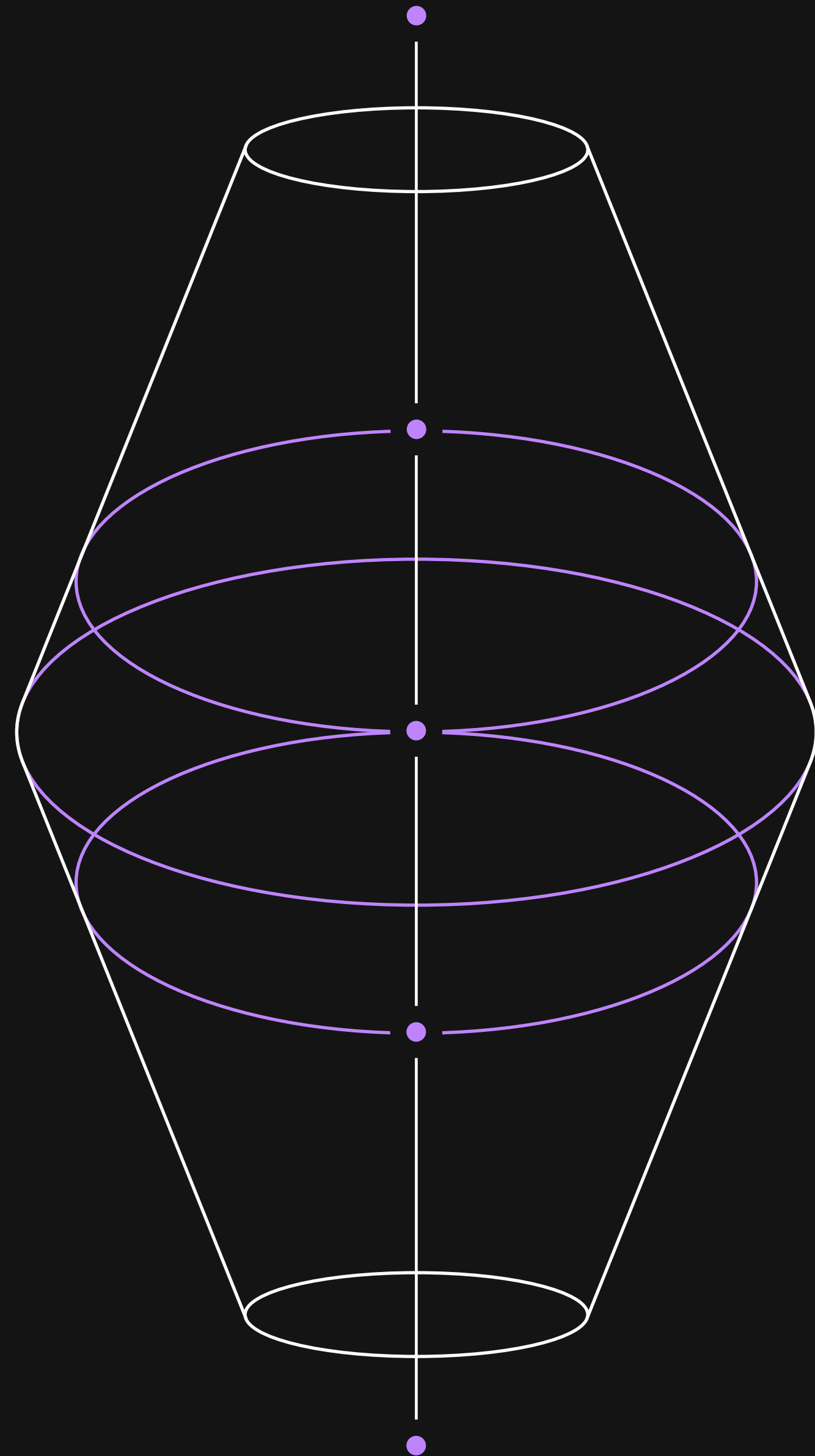
Группировка данных — это объединение строк таблицы на основе значений одного или нескольких столбцов (ключа группировки). Одинаковые значения ключа группировки при этом объединяются в одну строку.

Группировка обычно применяется в сочетании с функциями агрегации.



GROUP BY

Для группировки данных



- Синтаксис:
SELECT
 столбец1,
 столбец2,
 АГРЕГАТНАЯ_ФУНКЦИЯ(столбец3)
FROM таблица
 [WHERE условие_до_группировки]
GROUP BY столбец1, столбец2;
- Все столбцы в **SELECT**, не являющиеся частью агрегатных функций, должны быть в **GROUP BY**.
- **GROUP BY** можно использовать без агрегатных функций; равносильно **DISTINCT**.

GROUP BY



ПРИМЕР 6

Посчитать сумму и количество всех созданных заказов по каждому клиенту.

GROUP BY



ПРИМЕР 6

Посчитать сумму и количество всех созданных заказов по каждому клиенту.

• • •

```
SELECT
    customer_id,
    SUM(total_amount) AS total_sum,
    COUNT(*) AS orders_count
FROM shop_orders
GROUP BY customer_id
```


Вопрос на подумать

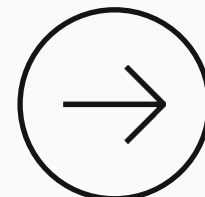


Какое минимальное и максимальное количество строк может получиться после группировки таблицы с N строками?

Вопрос на подумать



Какое минимальное и максимальное количество строк может получиться после группировки таблицы с N строками?



От одной до N .

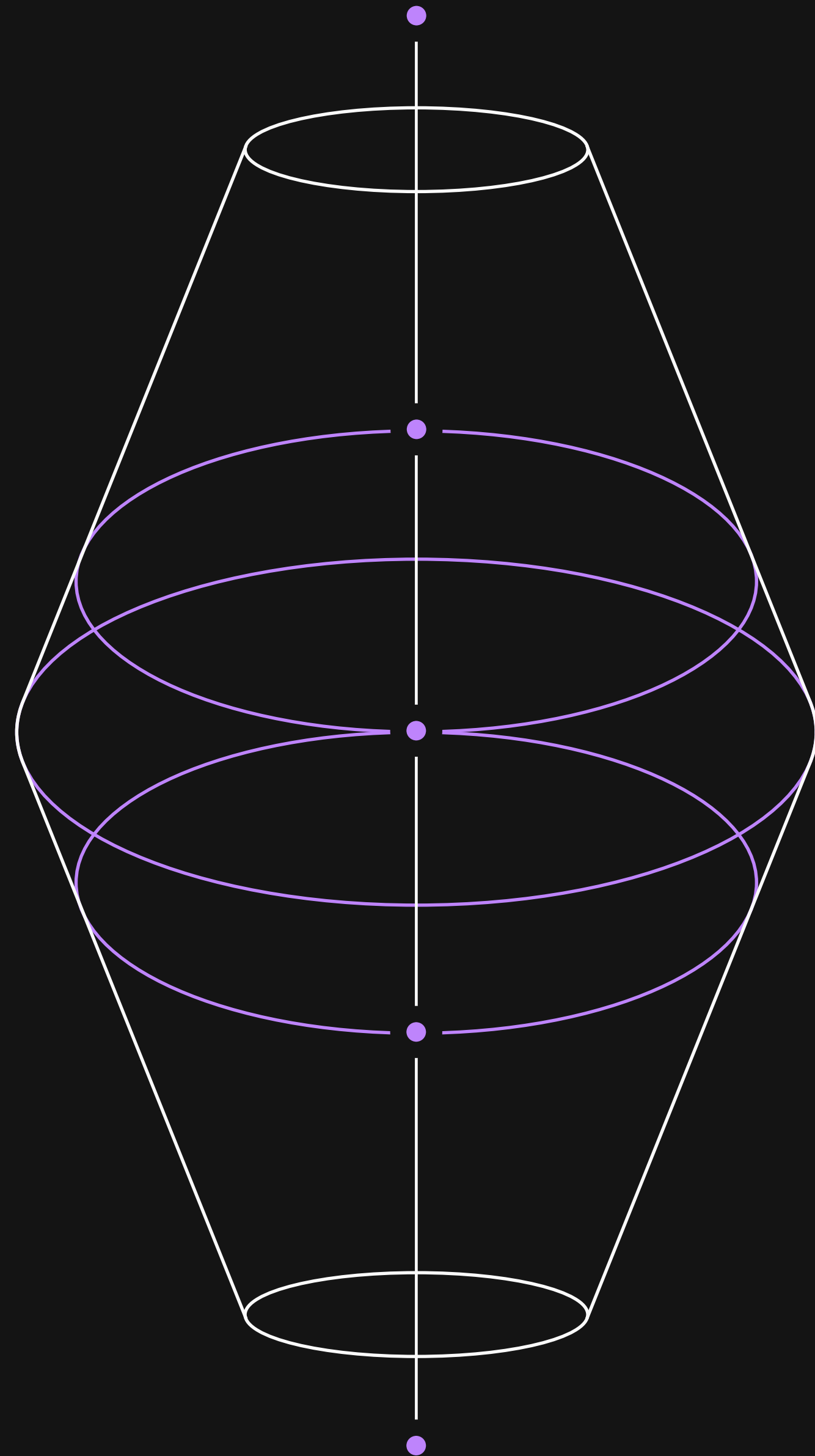
HAVING

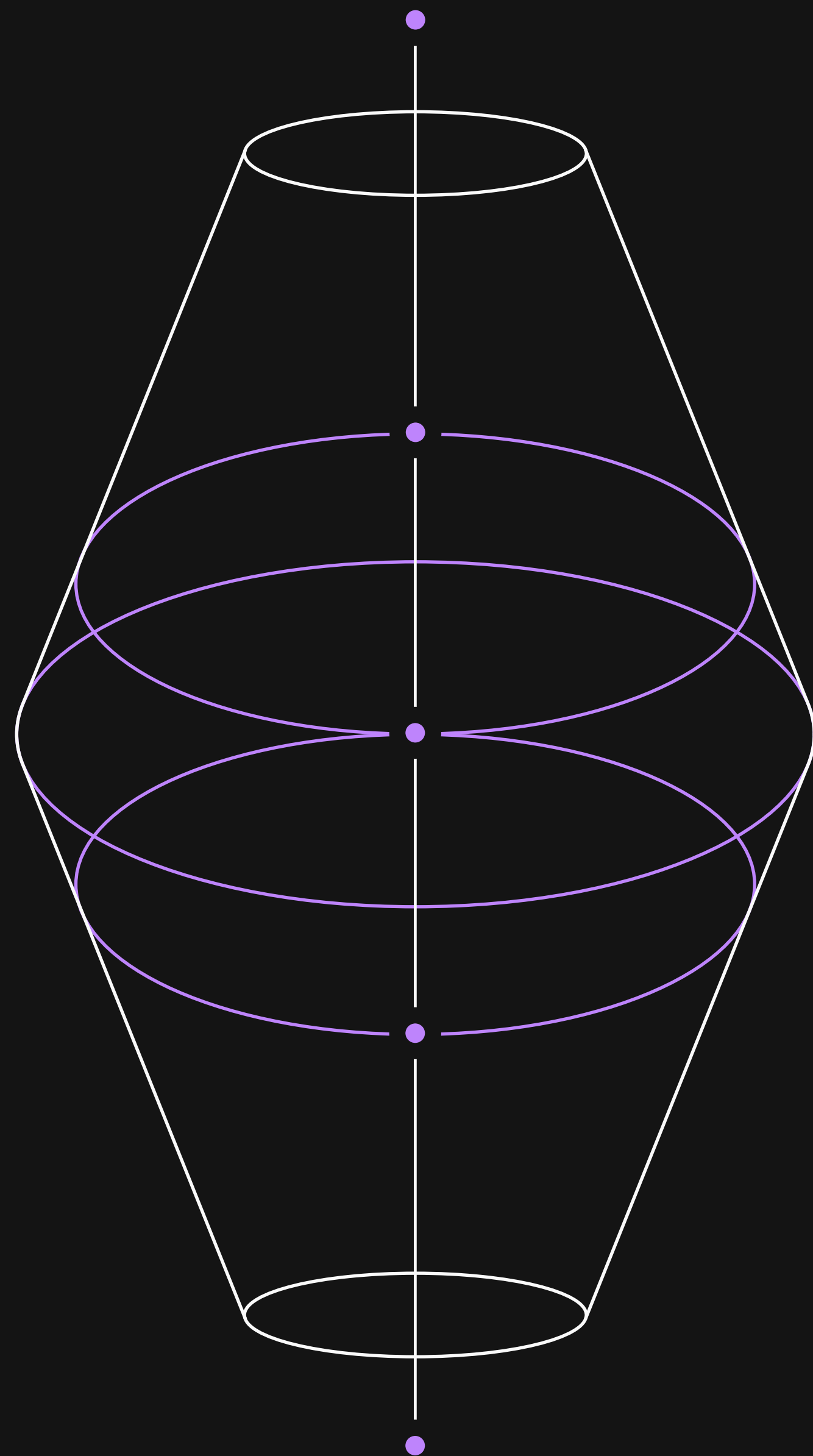
Для фильтрации сгруппированных данных

→ Синтаксис:

```
SELECT
    столбец1,
    столбец2,
    АГРЕГАТНАЯ_ФУНКЦИЯ(столбец3)
FROM таблица
[WHERE условие_до_группировки]
GROUP BY столбец1, столбец2
HAVING агрегатное_условие;
```

→ **HAVING** позволяет применить условие к результату группировки, а **WHERE** фильтрует строки до этого — в момент выбора данных из таблиц.





Выполнение запроса



Порядок фактического выполнения
не равен порядку написания в запросе.

Написание

Выполнение

SELECT

FROM

FROM

WHERE

WHERE

GROUP BY

GROUP BY

HAVING

HAVING

SELECT

ORDER BY

ORDER BY

LIMIT

LIMIT

Применение HAVING



ПРИМЕР 7

Найти клиентов, совершивших более десяти успешных (Completed) заказов.

Применение HAVING

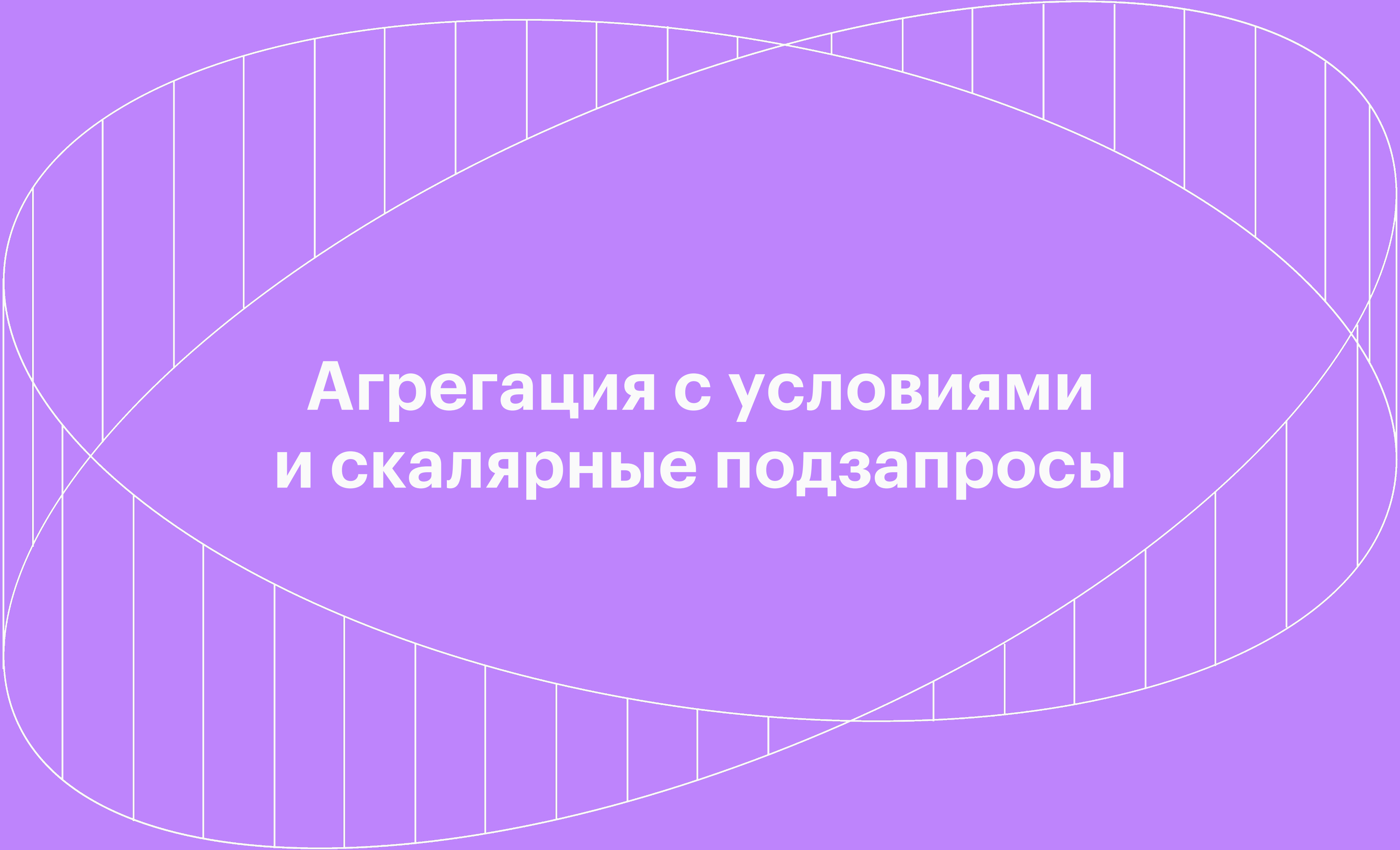


ПРИМЕР 7

Найти клиентов, совершивших более десяти успешных (Completed) заказов.

...

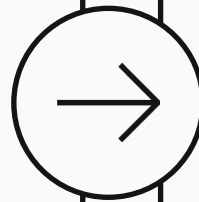
```
SELECT
    customer_id,
    SUM(total_amount) AS total_sum,
    COUNT(*) AS orders_count
FROM shop_orders
WHERE order_status = 'Completed'
GROUP BY customer_id
HAVING COUNT(*) > 10
```



Агрегация с условиями и скалярные подзапросы

Примеры использования агрегации в СЛОЖНЫХ ВЫЧИСЛЕНИЯХ

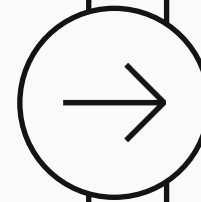
- Выборочное суммирование по столбцу
- Разделение суммы на несколько столбцов



Условная агрегация

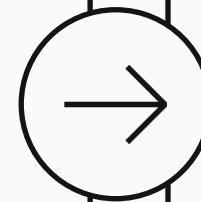
Примеры использования агрегации в сложных вычислениях

- Выборочное суммирование по столбцу
- Разделение суммы на несколько столбцов



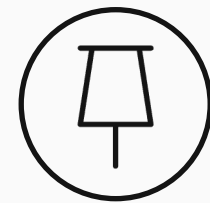
Условная агрегация

- Сравнение значения метрик с общей суммой
- Оценка отклонений показателей от среднего в группе



Скалярный подзапрос в SELECT

Условная агрегация



Иногда в одном запросе нужно получить несколько агрегированных значений по разным условиям. Для этого удобно использовать условную агрегацию.

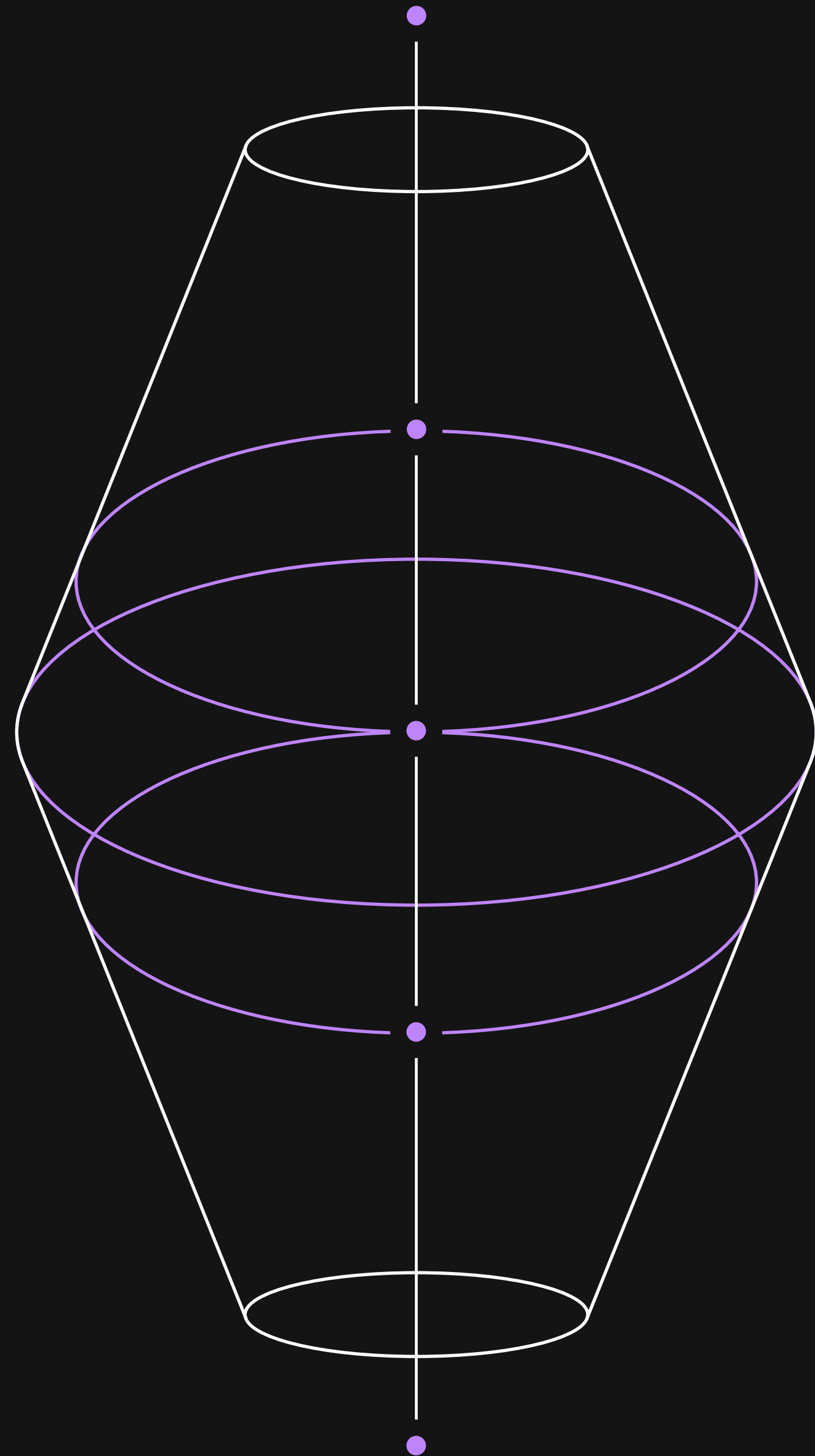
Например, с помощью условной агрегации можно найти сумму выполненных и сумму отменённых заказов, не разделяя данные на отдельные выборки.

Агрегация с условиями

Чтобы посчитать агрегированное значение только на части данных

- Синтаксис **FILTER**:

```
SELECT
    столбец1,
    АГРЕГАТНАЯ_ФУНКЦИЯ(столбец2)
    FILTER (WHERE условие) AS алиас
FROM таблица
GROUP BY столбец1;.
```
- В **WHERE** можно передавать не только колонки, но и более сложные составные условия.



Агрегация с условиями



ПРИМЕР 8

По каждому клиенту найди количество всех созданных когда-либо заказов, а также количество выполненных заказов, которые дороже 10 000.

Агрегация с условиями



ПРИМЕР 8

По каждому клиенту найди количество всех созданных когда-либо заказов, а также количество выполненных заказов, которые дороже 10 000.

...

```
SELECT
    customer_id,
    COUNT(*) AS all_orders,
    COUNT(*) FILTER (WHERE total_amount > 10000 AND order_status = 'Completed')
AS orders_count
FROM shop_orders
GROUP BY customer_id
```

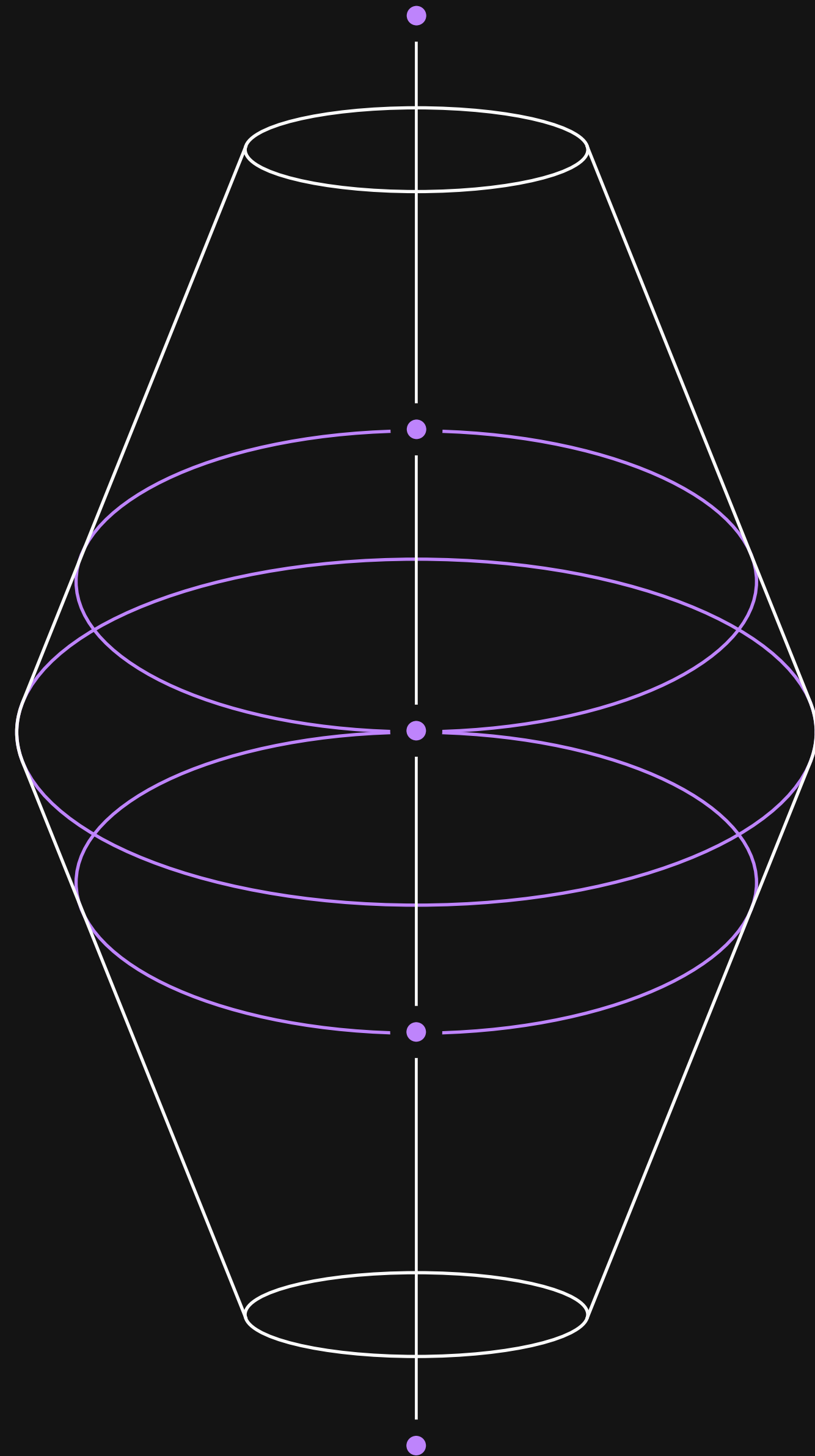

Агрегация с условиями

Чтобы посчитать агрегированное значение только на части данных

→ Синтаксис **CASE WHEN**:

```
SELECT
    АГРЕГАТНАЯ_ФУНКЦИЯ(
        CASE
            WHEN условие THEN значение1
            ELSE значение2
        END
    ) AS алиас
FROM таблица.
```

→ Может принимать несколько условий **WHEN... THEN**.



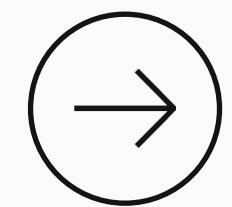
Агрегация с условиями



ПРИМЕР 9

Для каждого клиента найти общую сумму заказов, а также общий размер скидки по всем его заказам.

- Если заказ стоил меньше 100 000, то скидка равна 2% от стоимости заказа.
- Если больше или равно 100 000, то скидка равна 1% от стоимости заказа.



Попробуем решить задачу с помощью CASE в агрегирующей функции.

Агрегация с условиями



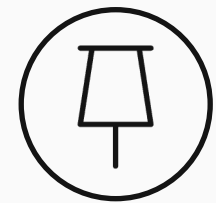
ПРИМЕР 9

Для каждого клиента найти общую сумму заказов, а также общий размер скидки по всем его заказам.

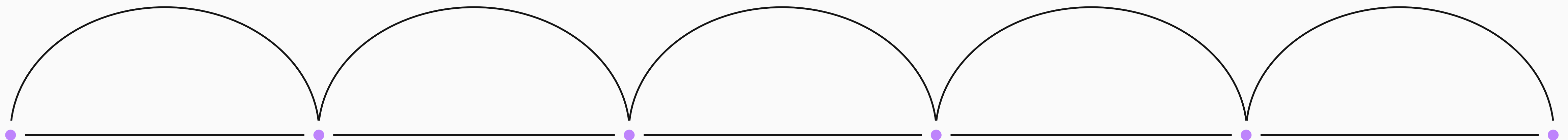
- Если заказ стоил меньше 100 000, то скидка равна 2% от стоимости заказа.
- Если больше или равно 100 000, то скидка равна 1% от стоимости заказа.

```
SELECT
    customer_id,
    SUM(total_amount) AS sum_amount,
    SUM(
        CASE
            WHEN total_amount < 100000
            THEN total_amount*0.02
            ELSE total_amount*0.01
        END
    ) AS discount
FROM shop_orders
GROUP BY customer_id
```

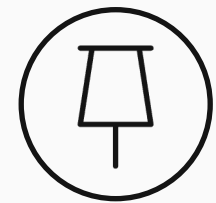

Ограничения агрегации



Нельзя использовать на одном уровне запроса детальные (неагрегированные) поля вместе с агрегированными, если эти детальные поля не указаны в секции GROUP BY. Для такой задачи удобно применять скалярные подзапросы в SELECT.

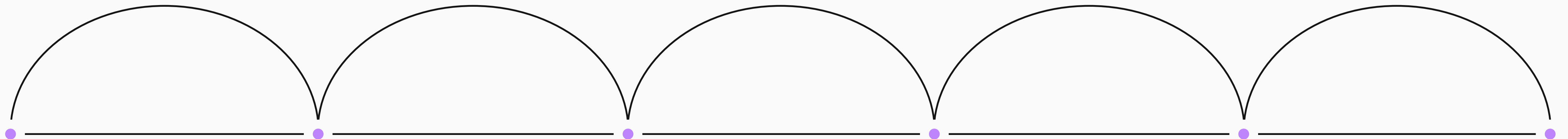


Ограничения агрегации



Скалярный подзапрос — это вложенный подзапрос, который возвращает ровно одно значение.

Вычисляется один раз (если он некоррелированный) или по каждой строке (если коррелированный). Полезен, если нужно сравнить значение строки с агрегатом.



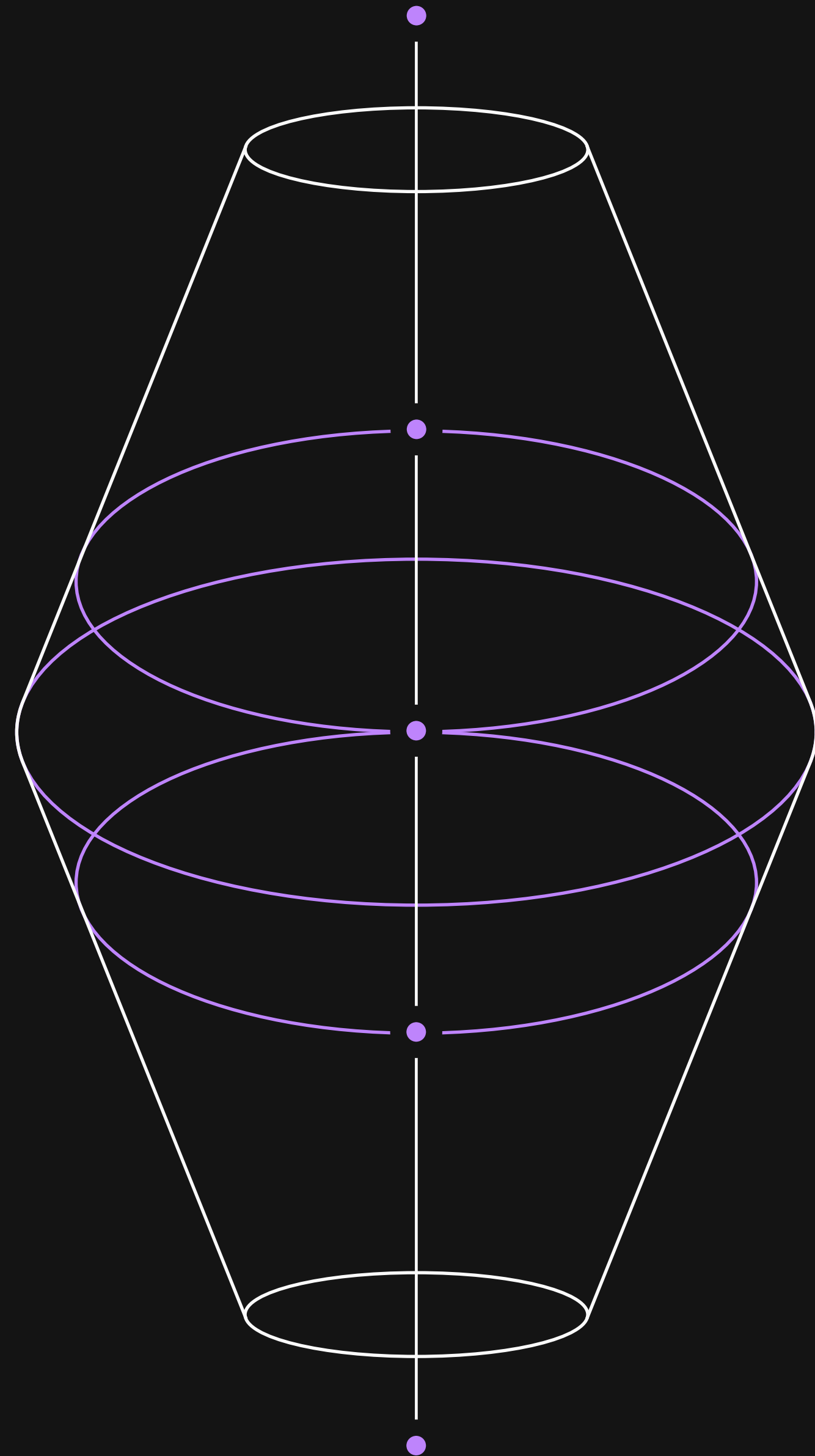
Некоррелированный подзапрос с агрегацией в **SELECT**

Чтобы получить одно агрегированное значение
в результате каждого вывода строки

→ Синтаксис

```
SELECT
    столбец1,
    столбец2,
    (
        SELECT АГРЕГАТНАЯ_ФУНКЦИЯ(столбец3)
        FROM другая_таблица
        WHERE условие
    ) AS алиас
FROM таблица;
```

→ Вычисление не зависит от конкретной строки.



Некоррелированный подзапрос с агрегацией в SELECT



ПРИМЕР 10

Сравнить сумму каждого заказа со средним значением этого показателя по всем данным.

Некоррелированный подзапрос с агрегацией в SELECT

ПРИМЕР 10

Сравнить сумму каждого заказа со средним значением этого показателя по всем данным.

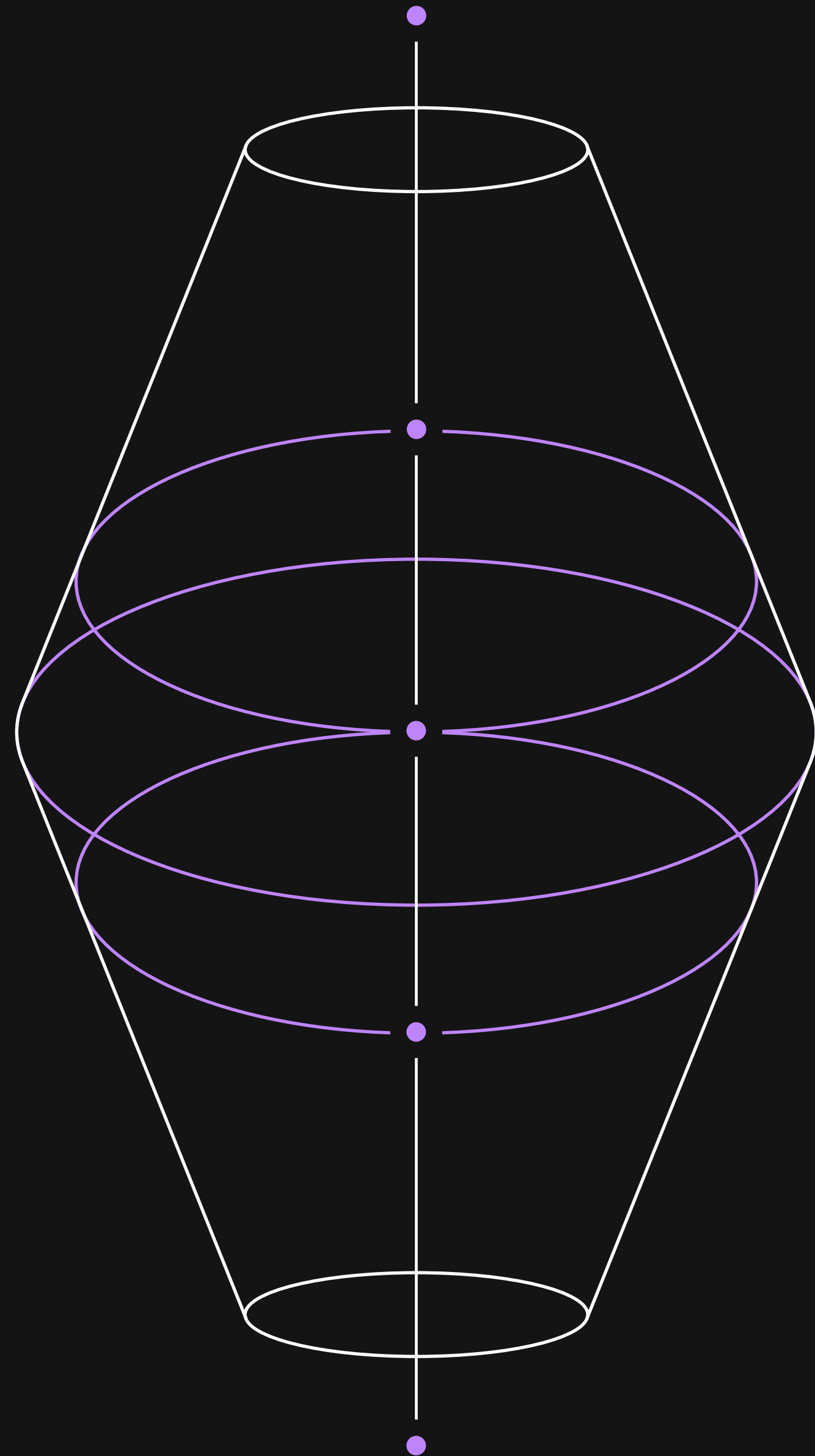
```
SELECT
    order_id,
    total_amount,
    (SELECT AVG(total_amount) FROM shop_orders) AS avg_sum,
    total_amount - (SELECT AVG(total_amount) FROM shop_orders) AS diff
FROM shop_orders
```


Коррелированный подзапрос с агрегацией в **SELECT**

Чтобы для каждой строки получить агрегированное значение внутри какой-то группы (клиента, товара и т. д.)

- Синтаксис


```
SELECT
  столбец1,
  столбец2,
  (
    SELECT АГРЕГАТНАЯ_ФУНКЦИЯ(таблица.столбец3)
    FROM таблица
    WHERE таблица.id_группы = внешняя_таблица.id_группы
  ) AS алиас
FROM внешняя_таблица;
```
- Вычисляется отдельно для каждой строки внешнего запроса.



Коррелированный подзапрос с агрегацией в SELECT

ПРИМЕР 11

Для каждого заказа нужно сравнить его стоимость со средней стоимостью заказов того же клиента.

В качестве ответа выведи следующие столбцы:

- `customer_id` — идентификатор клиента;
- `order_id` — идентификатор заказа;
- `total_amount` — стоимость заказа клиента;
- `avg_customer_amount` — средняя стоимость заказов клиента;
- `diff_avg_amount` — разница по модулю между стоимостью заказа и средней стоимостью заказов клиента.

Отсортируй результат по идентификатору клиента и номеру заказа.

Коррелированный подзапрос с агрегацией в SELECT

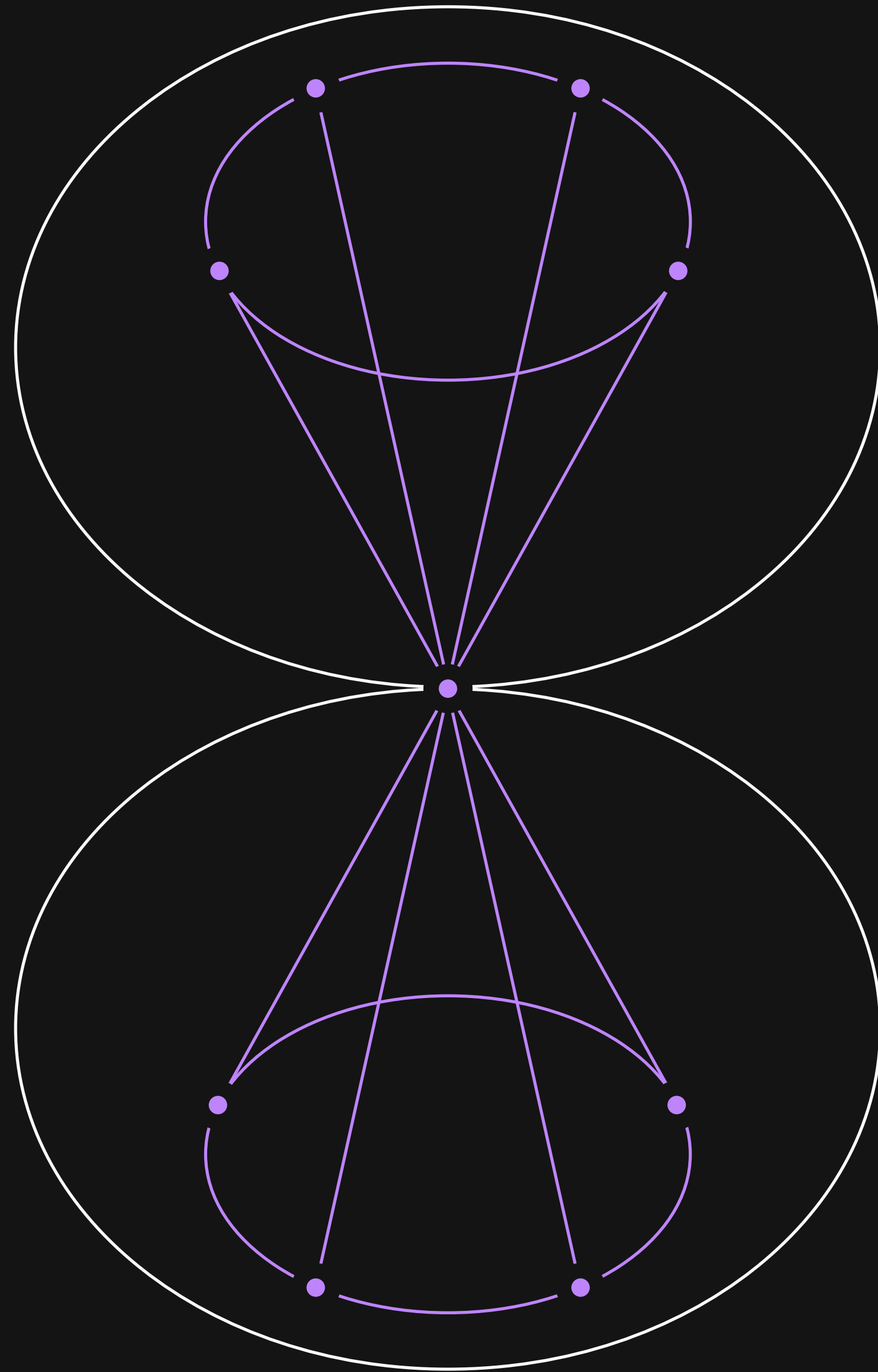
ПРИМЕР 11

Для каждого заказа нужно сравнить его стоимость со средней стоимостью заказов того же клиента.

Подзапросы

```
SELECT
  customer_id,
  order_id,
  total_amount,
  (
    SELECT AVG(total_amount) FROM shop_orders s1
    WHERE s1.customer_id = s2.customer_id
  ) AS avg_customer_amount,
  abs(total_amount - (
    SELECT AVG(total_amount)
    FROM shop_orders s1
    WHERE s1.customer_id = s2.customer_id)
  ) AS diff_avg_amount
FROM shop_orders s2
ORDER BY customer_id, order_id DESC;
```


Подведём итоги



→ Агрегирующие функции:

- `COUNT ()` ;
- `SUM ()` ;
- `MIN ()` , `MAX ()` ;
- `AVG ()` ;

Группировка: `GROUP BY`.

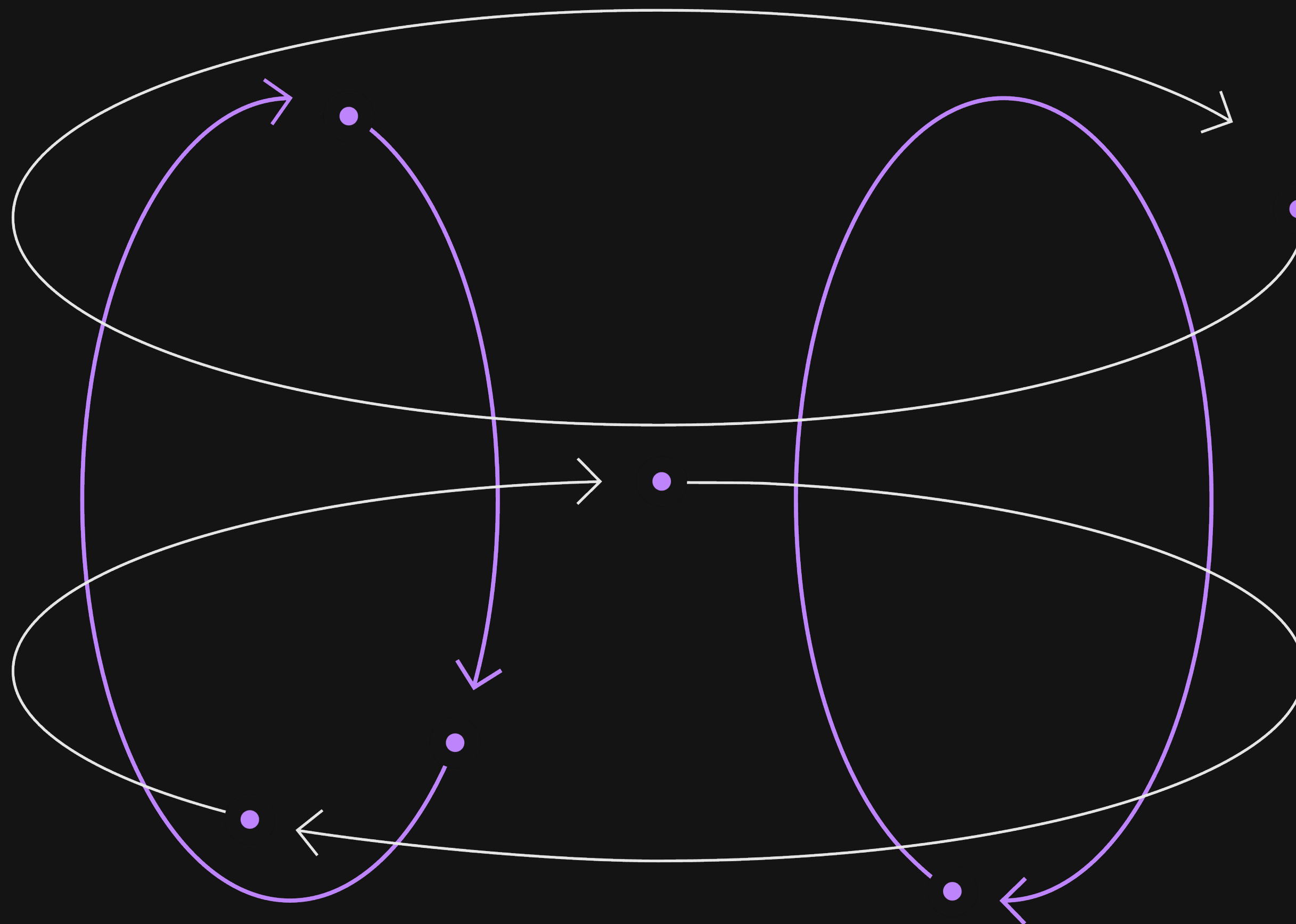
→ Фильтрация результатов группировки: `HAVING`.

→ Агрегация с условиями:

- агрегация() `FILTER (WHERE)` ;
- агрегация(`CASE WHEN ... THEN ... ELSE ... END`) .

→ Общий агрегат по всем данным или для конкретной строки можно добавить через подзапрос.

Вопросы



Минутка обратной связи

